

Informe módulo taller

Desarrollo y evaluación de **charruaLLM**:
un modelo de lenguaje autoregresivo entrenado con noticias uruguayas

Juan Pablo Conde, Bruno Turcatti

18 de agosto de 2026

Resumen

Este trabajo presenta el desarrollo y la evaluación de **charruaLLM**, un modelo de lenguaje Transformer autoregresivo de 17,1 millones de parámetros entrenado con UY22, el corpus de noticias uruguayas presentado junto con ROUBERTa. Para estudiar el efecto de la especialización de dominio se entrenó un segundo modelo, **spanishLLM**, con la misma arquitectura y un corpus general en español. Además, se construyó un conjunto de evaluación de 834 ejemplos a partir de afirmaciones de QuUANTO, derivadas originalmente de QuALES. Los modelos se compararon mediante exact matching de la palabra final y negative log-likelihood (NLL) de sus tokens. **charruaLLM** obtuvo 26,62 % de exact matching, frente a 4,68 % de **spanishLLM**, y una NLL media por palabra de 4,1396, frente a 22,8143. Los resultados muestran una ventaja clara del modelo especializado, en línea con los hallazgos de ROUBERTa, aunque deben interpretarse considerando las diferencias entre tokenizadores, capitalización y posible solapamiento de dominios.

Índice

1. Breves antecedentes	3
1.1. RoBERTa	3
1.2. ROUBERTa: antecedente directo sobre el corpus UY22	3
1.3. GPT-2	4
1.4. Chinchilla y elección de la cantidad de parámetros	5
1.5. Tokenización BPE	5
2. Descripción del trabajo	5
2.1. Arquitectura del modelo y cantidad de parámetros	6
2.2. Datos utilizados y procedencia	7
2.2.1. Corpus de <code>charruaLLM</code>	7
2.2.2. Corpus de <code>spanishLLM</code>	8
2.3. Creación del nuevo dataset de evaluación	8
2.4. Entrenamiento del modelo de comparación	9
2.5. Metodología de evaluación	9
2.5.1. Exact matching	9
2.5.2. NLL de la palabra objetivo	10
3. Resultados obtenidos	10
3.1. Gráfica de loss	10
3.2. Resultados de exact matching	11
3.3. Resultados de NLL	11
3.4. Análisis cualitativo de las predicciones	12
3.5. Interpretación general	12
3.6. Limitaciones y trabajo futuro	12
4. Conclusiones	13
5. Archivos producidos	13

1. Breves antecedentes

Los modelos de lenguaje modernos estiman la probabilidad de una secuencia de texto a partir de sus componentes. En un modelo autoregresivo, la probabilidad de una secuencia de tokens $t_1, \dots, t_n$ se factoriza como

$$P(t_1, \dots, t_n) = \prod_{i=1}^n P(t_i \mid t_1, \dots, t_{i-1}). \quad (1)$$

Esta formulación permite entrenar un modelo con texto sin etiquetar mediante una tarea simple: predecir el token siguiente. La arquitectura Transformer, introducida por Vaswani et al. [8], sustituyó la recurrencia de arquitecturas anteriores por mecanismos de atención. Esto permitió modelar dependencias entre posiciones de una secuencia y paralelizar mejor el entrenamiento.

A partir del Transformer surgieron dos grandes familias de modelos de lenguaje: los modelos de tipo encoder, como BERT y RoBERTa, y los modelos autoregresivos decoder-only, como GPT y GPT-2. El modelo desarrollado en este trabajo pertenece a la segunda familia. Su objetivo no es clasificar texto ni responder preguntas mediante fine-tuning, sino aprender una distribución sobre el texto y generar continuaciones mediante predicción del token siguiente.

1.1. RoBERTa

RoBERTa, cuyo nombre significa *A Robustly Optimized BERT Pretraining Approach*, es una revisión de la receta de entrenamiento de BERT presentada por Liu et al. [3]. El trabajo mostró que parte de las mejoras atribuidas a nuevas arquitecturas podía obtenerse entrenando BERT durante más tiempo, con lotes mayores y más datos. También eliminó el objetivo de *Next Sentence Prediction*, utilizó secuencias más largas y aplicó enmascaramiento dinámico.

RoBERTa es un modelo encoder y se entrena con *masked language modeling*: recibe información a ambos lados de una posición enmascarada. Por ello no es la arquitectura usada directamente en `charruaLLM`, que genera texto de izquierda a derecha. Sin embargo, es un antecedente relevante por dos motivos. Primero, muestra que los datos y el régimen de entrenamiento pueden importar tanto como los cambios arquitectónicos. Segundo, el sistema ganador de la tarea QuALES 2022 empleó un modelo RoBERTa multilingüe preentrenado y ajustado para preguntas y respuestas [5]. Esto aporta un punto de referencia para el origen del dataset que luego adaptamos a nuestra evaluación de completado de palabras.

1.2. ROUBERTa: antecedente directo sobre el corpus UY22

El antecedente más cercano a este trabajo es ROUBERTa, presentado por Filevich et al. [1] en *A Language Model Trained on Uruguayan Spanish News Text*. El nombre combina RoBERTa con ROU, sigla de República Oriental del Uruguay. El artículo introdujo UY22, un corpus específico de prensa uruguaya, y entrenó desde cero variantes *cased* y *uncased* de un modelo RoBERTa sobre su versión limpia.

UY22 reúne aproximadamente 900.000 artículos, 400 millones de palabras y más de 6 GiB de texto sin comprimir, con noticias publicadas desde comienzos de la década del 2000 hasta diciembre de 2022. Sus cuatro fuentes son El Observador, El País, Montevideo Portal y La Diaria. El proceso de limpieza incluyó la eliminación de HTML y espacios duplicados, normalización de caracteres, eliminación de emojis, reemplazo de enlaces y descarte de artículos con menos de 16 palabras.

ROUBERTa utiliza una arquitectura encoder de tipo RoBERTa-base, un tokenizer BPE de 30.000 tokens y dynamic masked language modeling con una probabilidad de enmascaramiento del 15 %. Fue entrenado durante 30 días y unos seis millones de pasos en una NVIDIA P100 de 12 GiB. El entrenamiento comenzó con una longitud máxima de 384 tokens y luego continuó con longitud 128 y batch size 32 para ajustarse a la memoria disponible. La versión pública tiene un orden de magnitud de 100 millones de parámetros¹, por lo que sigue siendo considerablemente mayor que los 17,1 millones de parámetros de `charruaLLM`.

El artículo evaluó ROUBERTa en análisis de sentimiento, Question Answering y predicción de palabras enmascaradas. La variante cased alcanzó 75,0 % de accuracy en sentimiento y, después de fine-tuning sobre QuALES, obtuvo 28,1 de Exact Match y 32,3 de F1. En siete textos periodísticos recientes y no vistos, predijo correctamente 982 de 2.368 palabras enmascaradas, un 42 % de top-1, frente al 27 % obtenido tanto por BETO como por XLM-RoBERTa.

Dos ablaciones del artículo son especialmente relevantes para nuestro experimento. Primero, un RoBERTa entrenado con 4 GiB de UY22 alcanzó 68,6 % en sentimiento, frente a 65,0 % de otro modelo entrenado con 20 GiB del corpus general usado por BETO. Segundo, la variante cased alcanzó 75,0 %, frente a 68,6 % de la variante uncased. Estos resultados respaldan tanto la hipótesis de especialización de dominio como la importancia de la capitalización que observamos al comparar `charruaLLM` con `spanishLLM`.

ROUBERTa y `charruaLLM` comparten el corpus de origen y la motivación de crear modelos uruguayos utilizables con recursos limitados, pero resuelven objetivos distintos. ROUBERTa es bidireccional: al completar una máscara puede utilizar el texto ubicado a ambos lados de la palabra. `charruaLLM` es causal y solo puede usar el contexto anterior. Por ese motivo, el 42 % de masked-word prediction de ROUBERTa y el 26,62 % de exact matching de nuestro modelo no deben interpretarse como una comparación directa de calidad.

Cuadro 1: Comparación conceptual entre ROUBERTa y `charruaLLM`.

Aspecto	ROUBERTa	<code>charruaLLM</code>
Corpus de origen	UY22	UY22, versión procesada local
Arquitectura	Transformer encoder, RoBERTa	Transformer decoder-only
Objetivo	Masked language modeling	Predicción causal del token siguiente
Contexto disponible	Izquierda y derecha de la máscara	Solo tokens anteriores
Vocabulario	BPE, 30.000 tokens	Byte-Level BPE, 16.384 tokens
Parámetros	Aproximadamente 0,1B	17.114.880
Pasos reportados	Aproximadamente 6 millones	960.000

1.3. GPT-2

GPT-2 fue presentado por Radford et al. [4] en *Language Models are Unsupervised Multitask Learners*. Es un Transformer decoder-only entrenado con un objetivo causal: predecir cada token a partir de los tokens anteriores. El trabajo mostró que, al aumentar la escala y la diversidad de los datos, un mismo modelo autoregresivo podía adquirir capacidades útiles para diferentes tareas sin modificar sus parámetros para cada una de ellas.

`charruaLLM` sigue esta idea central a una escala mucho menor y orientada al español uruguayo. Al igual que GPT-2, utiliza atención causal, bloques Transformer decoder-only, embeddings posicionales aprendidos y generación autoregresiva. La diferencia principal está en la escala: el GPT-2 más grande tenía 1.500 millones de parámetros, mientras que cada modelo de este trabajo

¹Ficha del modelo: <https://huggingface.co/pln-udelar/ROUBERTa-base-uy22-cased>.

tiene 17.114.880 parámetros. Nuestro objetivo fue construir un modelo que pudiera entrenarse con recursos académicos accesibles y estudiar el efecto del dominio de los datos, no reproducir la escala de GPT-2.

1.4. Chinchilla y elección de la cantidad de parámetros

Hoffmann et al. [2], en *Training Compute-Optimal Large Language Models*, estudiaron cómo distribuir un presupuesto de cómputo entre la cantidad de parámetros del modelo y la cantidad de tokens de entrenamiento. El resultado principal fue que ambos factores deben crecer de forma aproximadamente pareja. El trabajo mostró que varios modelos grandes estaban subentrenados: tenían demasiados parámetros para la cantidad de datos que habían procesado. Como regla práctica derivada del estudio suele mencionarse un orden de magnitud de aproximadamente 20 tokens de entrenamiento por parámetro, aunque no debe interpretarse como una constante universal, especialmente fuera de las escalas analizadas en el artículo.

Cada modelo de este proyecto tiene 17.114.880 parámetros. El corpus uruguayo contiene 523.695.440 tokens en train, equivalentes a 30,60 tokens únicos de train por parámetro. El corpus general en español contiene 735.225.214 tokens en train, equivalentes a 42,96 tokens por parámetro. Estos cocientes muestran que ambos corpus poseen suficiente volumen para el tamaño elegido.

No obstante, durante 960.000 pasos, con batches de 32 secuencias de 512 tokens, cada entrenamiento presenta aproximadamente 15.728.640.000 posiciones de token al modelo. Esto implica múltiples recorridos estadísticos sobre el corpus: unas 30 veces el tamaño del train uruguayo y unas 21 veces el del train general. Por lo tanto, el experimento no debe describirse como una reproducción estricta del óptimo de Chinchilla. La decisión práctica fue mantener un modelo pequeño y entrenarlo extensamente. Con un presupuesto fijo de cómputo, el trabajo de Chinchilla sugiere que también sería pertinente experimentar con un modelo de mayor tamaño y menos repeticiones de los datos.

1.5. Tokenización BPE

Los dos modelos utilizan tokenizadores Byte-Level BPE independientes, con un vocabulario de 16.384 tokens y el token especial `<|endoftext|>`. BPE representa palabras frecuentes mediante uno o pocos tokens y descompone palabras poco frecuentes en unidades menores [7]. Esto evita depender de un vocabulario cerrado de palabras completas y permite representar nombres propios, variaciones morfológicas y caracteres acentuados.

La elección de tokenizadores independientes permite adaptar el vocabulario a cada corpus, pero introduce una precaución al comparar métricas por token. Una misma palabra puede tener diferente cantidad de tokens en cada modelo. En la evaluación realizada, las 834 palabras objetivo produjeron 1.467 tokens con el tokenizer de `charruaLLM` y 2.553 con el de `spanishLLM`.

2. Descripción del trabajo

El objetivo fue construir un modelo de lenguaje base para noticias uruguayas en español y medir si la especialización de dominio le permite anticipar mejor la última palabra de afirmaciones relacionadas con noticias uruguayas. Para que la comparación no dependiera únicamente del tamaño o de la arquitectura, también se entrenó un segundo modelo con exactamente la misma configuración, pero con un corpus amplio y heterogéneo en español.

El flujo de trabajo fue el siguiente:

1. Obtención y preparación del corpus de noticias uruguayas UY22.
2. Entrenamiento de un tokenizer Byte-Level BPE específico.
3. Conversión del corpus a secuencias de identificadores de tokens.
4. Separación por documentos en train, validation y test.
5. Entrenamiento de **charruaLLM** mediante predicción del token siguiente.
6. Preparación de un corpus general en español y entrenamiento de **spanishLLM**.
7. Creación de un dataset de evaluación, QuUANTO, con afirmaciones derivadas de pares pregunta-respuesta en QuALES.
8. Evaluación mediante exact matching y negative log-likelihood (NLL).
9. Almacenamiento de las predicciones y métricas en archivos CSV.

2.1. Arquitectura del modelo y cantidad de parámetros

Ambos modelos comparten la arquitectura resumida en la Tabla 2.

Cuadro 2: Configuración arquitectónica de los dos modelos.

Componente	Configuración
Tipo	Transformer autoregresivo decoder-only
Capas Transformer	6
Cabezas de atención	6 por capa
Dimensión del embedding	384
Dimensión interna del MLP	1.536
Contexto máximo	512 tokens
Vocabulario	16.384 tokens
Posiciones	Embeddings aprendidos
Activación	GELU
Normalización	Pre-norm con LayerNorm
Atención	Scaled dot-product attention causal
Dropout	0,0
Capa de salida	Pesos compartidos con el embedding de tokens

Cada bloque utiliza conexiones residuales en los subbloques de atención y MLP. La distribución exacta de parámetros se muestra en la Tabla 3.

Cuadro 3: Distribución de parámetros del modelo.

Componente	Parámetros
Embedding de tokens	6.291.456
Embedding de posiciones	196.608
Seis bloques Transformer	10.626.048
LayerNorm final	768
Total	17.114.880

El peso compartido entre el embedding y la salida evita agregar otra matriz de 16.384×384 parámetros. Todos los parámetros son entrenables.

La configuración de entrenamiento fue la misma para los dos modelos:

- Optimizador AdamW, con $\beta_1 = 0,9$ y $\beta_2 = 0,95$.
- Learning rate máximo de 3×10^{-4} y mínimo de 3×10^{-5} .
- Warmup de 200 pasos y decaimiento cosenoidal.
- Batch size de 32 y gradient accumulation igual a 1.
- Gradient clipping de 1,0 y weight decay de 0,1.
- 960.000 pasos de entrenamiento.
- Evaluación cada 250 pasos sobre 50 batches.
- Semilla de entrenamiento 1337.
- Cross-entropy para la predicción causal del token siguiente.

2.2. Datos utilizados y procedencia

2.2.1. Corpus de charruaLLM

El corpus utilizado pertenece a UY22, el mismo recurso presentado en el trabajo de ROUBERTa [1]. El corpus original comprende El Observador, El País, Montevideo Portal y La Diaria. La distribución publicada por sus autores se muestra en la Tabla 4.

Cuadro 4: Composición del corpus UY22 reportada por Filevich et al.

Medio	Artículos	Palabras	Participación
El Observador	314.821	150.007.925	37 %
El País	147.004	92.605.424	23 %
Montevideo Portal	433.244	145.422.666	36 %
La Diaria	20.000	14.079.916	4 %
Total	915.069	402.115.931	100 %

La metadata del entrenamiento reproducido en este repositorio registra tres archivos procesados, correspondientes a El Observador, El País y Montevideo Portal:

- eo22-unified-splitted.txt;
- ep22-unified-splitted.txt;
- mp22-unified-splitted.txt.

Las líneas no vacías se agruparon como documentos y los documentos se separaron de forma reproducible con semilla 42. La partición resultante se muestra en la Tabla 5.

En consecuencia, el origen de los datos es UY22, pero la versión procesada que figura en el checkpoint local contiene tres de sus cuatro medios y no registra un archivo de La Diaria. La diferencia entre las aproximadamente 402 millones de palabras informadas en el artículo y los 523,7 millones de tokens de nuestro train se debe, además, a que una palabra puede convertirse en varios tokens BPE.

Cuadro 5: Particiones de los corpus utilizados.

Modelo	Split	Documentos	Tokens
charruaLLM	Train	884.682	523.695.440
charruaLLM	Validation	4.468	2.658.412
charruaLLM	Test	4.468	2.707.620
spanishLLM	Train	250.929	735.225.214
spanishLLM	Validation	1.263	712.293
spanishLLM	Test	1.304	808.811

Se reservó 0,5 % de los documentos para validation y 0,5 % para test. El token `<|endoftext|>` se agregó al final de cada documento.

2.2.2. Corpus de spanishLLM

Para construir un control de español general se emplearon muestras preprocesadas y convertidas a minúsculas de 15 corpus que corresponden a la composición de Large Spanish Corpus [9]: DGT, DOGC, ECB, EMEA, EUBookShop, Europarl, GlobalVoices, JRC-Acquis, News Commentary 11, OpenSubtitles 2018, ParaCrawl, TED, UN, Spanish Wikis y MultiUN. Muchos de estos recursos provienen de la colección OPUS [10] y cubren dominios muy distintos: legislación, documentos institucionales, medicina, noticias, subtítulos, Wikipedia y transcripciones.

El train se dividió en 270.579 chunks y el material de origen ocupaba 3,0 GiB. Debido a su tamaño, se preparó mediante streaming, asignación determinista de documentos a cada split y tokenización en chunks acotados, sin cargar el corpus completo en memoria.

2.3. Creación del nuevo dataset de evaluación

Para evaluar el modelo se empleó QuUANTO, acrónimo de *QuALES-derived Uruguay Assertions for Neural Training Objectives*. Es un dataset de afirmaciones declarativas en español derivadas de pares de pregunta-respuesta de QuALES. QuALES fue una tarea de *Question Answering* extractivo organizada en IberLEF 2022 y basada principalmente en noticias uruguayas sobre la pandemia de COVID-19 y temas relacionados [5]. Las afirmaciones para la evaluación se tomaron del archivo `QuUANTO_unified_final.json` y del repositorio que documenta su construcción [6].

La construcción de QuUANTO combinó generación automática y revisión humana. En primer lugar, un modelo de lenguaje convirtió cada par de pregunta y respuesta en una afirmación autocontenida y marcó si su contenido dependía de un momento específico. Luego se conservaron las afirmaciones no sensibles al tiempo y se intentaron reescribir las restantes para expresar una formulación atemporal. También se aplicaron filtrado de contenido relacionado principalmente con COVID-19 y correcciones del orden de la información. Finalmente, las afirmaciones fueron sometidas a revisión humana. Este proceso mejora la coherencia y la fidelidad respecto de la fuente, pero no equivale a una verificación independiente de los hechos: el dataset hereda el sesgo temático y periodístico de las noticias de QuALES.

Para convertir las afirmaciones en una tarea de completado se tomó la última palabra de cada oración como objetivo y el resto como entrada. Por ejemplo:

Afirmación: El Pereira Rossell es un hospital.
Entrada: El Pereira Rossell es un
Objetivo: hospital

Se eliminó el punto final de la palabra objetivo y se mantuvo un espacio al principio de `goal` para conservar correctamente el límite de palabra al usar tokenizers Byte-Level. El resultado se guardó en `final_dataset.csv`, que contiene 834 ejemplos. Aunque cada afirmación conserva el split original de QuALES como dato de procedencia, para este trabajo se combinaron los ejemplos de `train`, `dev_gold` y `test_gold` en un único conjunto de evaluación. Por lo tanto, esos nombres no representan particiones propias del experimento ni se utilizaron para definir un conjunto held-out. Esta transformación evalúa conocimiento léxico y capacidad de continuación, pero ya no constituye la tarea extractiva de QA original.

2.4. Entrenamiento del modelo de comparación

`spanishLLM` se entrenó como modelo de control. Tiene exactamente la misma arquitectura, cantidad de parámetros, tamaño de vocabulario, longitud de contexto, optimizador y número de pasos que `charruaLLM`. Las variables que cambian son el corpus y el tokenizer aprendido sobre ese corpus.

Este diseño permite estudiar la hipótesis principal: ante una capacidad de modelo comparable, el entrenamiento sobre noticias uruguayas debería mejorar la predicción de palabras en afirmaciones del mismo dominio frente a un corpus general de español. Al mismo tiempo, `spanishLLM` fue entrenado con texto convertido a minúsculas, mientras el dataset de evaluación conserva mayúsculas y numerosos nombres propios. Esta diferencia forma parte del experimento, pero también debe considerarse al interpretar los resultados.

2.5. Metodología de evaluación

2.5.1. Exact matching

Para cada entrada se realizó generación greedy autoregresiva: en cada posición se eligió el token de mayor probabilidad. La generación se detuvo al detectar el comienzo de una segunda palabra, el token de fin de texto o un máximo de 32 tokens. La predicción se decodificó y se eliminó, de forma consistente con el dataset, un posible punto final.

El resultado se consideró correcto solamente cuando la palabra predicha fue idéntica a la palabra objetivo. La comparación fue sensible a mayúsculas, acentos y caracteres. Esta métrica es estricta:

una palabra semánticamente razonable, pero diferente del objetivo, cuenta como error. El detalle se guardó en `exact_matching_results.csv`; cada fila incluye el modelo, la afirmación, la entrada, el objetivo, la palabra predicha, los tokens generados y el indicador `exact_match`.

2.5.2. NLL de la palabra objetivo

La segunda evaluación midió la probabilidad que el modelo asignaba a la secuencia correcta de tokens, aunque esa secuencia no fuera la elegida por generación greedy. Se utilizó teacher forcing: para predecir cada token objetivo se proporcionaron como contexto los tokens objetivo anteriores correctos.

Para una palabra tokenizada como $t_1, \dots, t_k$, la métrica fue

$$\text{NLL}_{\text{palabra}} = - \sum_{i=1}^k \log P(t_i \mid c, t_1, \dots, t_{i-1}), \quad (2)$$

donde c representa el contexto anterior a la palabra. Una NLL menor indica que el modelo asignó mayor probabilidad al objetivo. La probabilidad conjunta de la palabra es

$$P(\text{palabra} \mid c) = \exp(-\text{NLL}_{\text{palabra}}). \quad (3)$$

También se guardaron la probabilidad y la NLL individual de cada token, la NLL media por token, la media geométrica de probabilidades y la perplexity por token. El detalle se almacenó en `nll_results.csv`, que también incluye la palabra generada en la evaluación de exact matching.

3. Resultados obtenidos

3.1. Gráfica de loss

La Figura 1 presenta, para cada modelo, la loss de entrenamiento y la loss de validación en función de los pasos. Los dos paneles utilizan la misma escala lineal en ambos ejes. La serie de train se muestra tanto en su forma original como suavizada mediante una media móvil de 200 observaciones, mientras que validation se muestra en los puntos de evaluación.

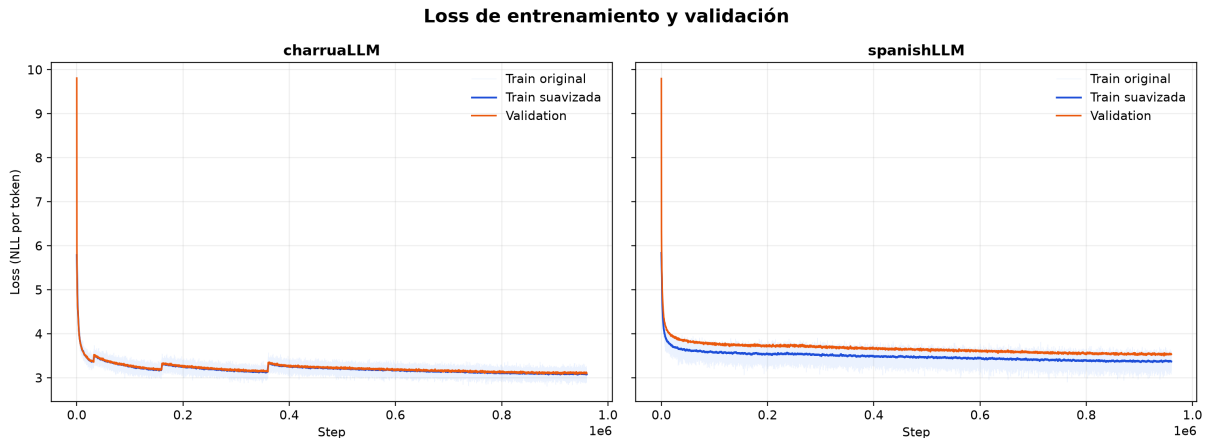


Figura 1: Loss de entrenamiento y validación de `charruaLLM` y `spanishLLM` hasta el paso 960.000.

Cuadro 6: Resumen numérico de las curvas de validación.

Modelo	Val inicial	Mejor val	Paso del mejor	Val final
charruaLLM	9,8118	3,0833	939.250	3,1035
spanishLLM	9,8010	3,5004	946.250	3,5364

La loss de train evaluada en el paso 960.000 fue 3,0924 para **charruaLLM** y 3,3738 para **spanishLLM**. Las dos curvas descienden fuertemente desde valores cercanos a 9,8 y luego se estabilizan. **charruaLLM** alcanza una loss de validación menor. En ambos casos la loss final es levemente superior al mínimo, por lo que para la evaluación se utilizaron los pesos **best.pt**: paso 939.250 para **charruaLLM** y paso 946.250 para **spanishLLM**.

3.2. Resultados de exact matching

Cuadro 7: Exact matching sobre los 834 ejemplos.

Modelo	Aciertos	Total	Exact matching
charruaLLM	222	834	26,62 %
spanishLLM	39	834	4,68 %

En el conjunto completo, **charruaLLM** supera a **spanishLLM** por 21,94 puntos porcentuales y obtiene aproximadamente 5,69 veces su tasa de acierto.

3.3. Resultados de NLL

Cuadro 8: Resultados agregados de NLL sobre los 834 ejemplos.

Modelo	NLL/palabra	NLL/token	Perplexity/token	Prob. palabra
charruaLLM	4,1396	2,3534	10,52	25,06 %
spanishLLM	22,8143	7,4529	1.724,80	3,00 %

La cantidad total de tokens objetivo fue 1.467 para **charruaLLM** y 2.553 para **spanishLLM**, equivalentes a 1,76 y 3,06 tokens por palabra respectivamente.

charruaLLM asigna mucha más probabilidad a las palabras correctas y obtiene una NLL sustancialmente menor. El resultado es consistente con exact matching, pero la NLL aporta información adicional: permite observar si el objetivo tenía una probabilidad razonable incluso cuando otra palabra quedó en primer lugar.

La comparación de NLL por token y perplexity requiere cautela porque cada modelo utiliza su propio tokenizer. **spanishLLM** divide las palabras objetivo en muchos más tokens, en parte porque fue entrenado con texto en minúsculas y el dataset contiene nombres propios y mayúsculas. La NLL total por palabra compara la probabilidad asignada a la misma cadena objetivo y es una referencia más interpretable entre los dos sistemas, pero tampoco separa por sí sola el efecto del modelo del efecto de su tokenizer. Una evaluación futura podría sumar bits por byte o bits por carácter para reducir esta dependencia.

3.4. Análisis cualitativo de las predicciones

La comparación exacta no captura por completo la calidad lingüística de las continuaciones. Algunos errores de **charruaLLM** son palabras relacionadas con el objetivo y resultan razonables en el contexto. Por ejemplo, ante la afirmación “El Pereira Rossell es un hospital”, el modelo predijo “centro”; y ante oraciones cuyo objetivo era “desempleo”, predijo “paro”. Estas respuestas no coinciden con la cadena esperada y por eso cuentan como errores de exact matching, pero muestran que el modelo puede recuperar relaciones léxicas y semánticas del dominio periodístico.

Este tipo de casos justifica complementar la métrica exacta con NLL. La primera responde si el modelo generó exactamente la palabra de referencia, mientras que la segunda permite medir cuánta probabilidad asignó a esa palabra aunque otra continuación haya quedado en primer lugar. El análisis también debe considerar que **spanishLLM** fue entrenado con texto en minúsculas, mientras que las afirmaciones conservan mayúsculas y nombres propios. Por ello, algunos errores pueden reflejar una diferencia de representación además de una falta de conocimiento del contenido.

3.5. Interpretación general

Los resultados apoyan la hipótesis de que la especialización de dominio es importante. Con la misma arquitectura y cantidad de parámetros, el modelo entrenado con noticias uruguayas predice mejor palabras de afirmaciones basadas en noticias uruguayas que el modelo entrenado con un corpus general en español. Esta conclusión es consistente con ROUBERTa, que también obtuvo mejores resultados con UY22 que con un corpus general cinco veces mayor y superó a BETO y XLM-RoBERTa en su evaluación de palabras enmascaradas no vistas.

La diferencia puede explicarse por varios factores relacionados:

1. **charruaLLM** fue entrenado con el dominio periodístico uruguayo.
2. Su tokenizer representa con menos tokens las palabras evaluadas.
3. El corpus general mezcla dominios muy heterogéneos.
4. **spanishLLM** fue entrenado con texto en minúsculas, mientras la evaluación conserva capitalización y numerosos nombres propios.
5. Los modelos son pequeños, por lo que la selección del dominio y el uso del vocabulario tienen un impacto particularmente visible.

Estos resultados no demuestran que un modelo general de español sea intrínsecamente inferior. Muestran que, bajo esta arquitectura, estos datos y este protocolo, el corpus especializado ofrece una ventaja clara para el benchmark seleccionado.

3.6. Limitaciones y trabajo futuro

- No se realizó una búsqueda sistemática de hiperparámetros. Sería útil comparar distintas tasas de aprendizaje, tamaños de modelo, longitudes de contexto y regímenes de entrenamiento.
- Sería conveniente analizar cualitativamente una muestra amplia de errores y agruparlos, por ejemplo, en sinónimos, nombres propios, errores morfológicos, errores sintácticos y falta de conocimiento.

- Sería útil entrenar con el dataset convertido a minúsculas para coincidir con el otro modelo.
- También podría reportarse una métrica auxiliar que considere equivalentes las predicciones que solo difieren en mayúsculas y minúsculas. La métrica principal debería mantenerse sensible a la capitalización para conservar la evaluación original.

4. Conclusiones

Se construyeron y entrenaron dos modelos Transformer decoder-only de 17,1 millones de parámetros: **charruaLLM**, especializado en noticias uruguayas, y **spanishLLM**, entrenado con un corpus general en español. Además, se creó un nuevo dataset de completado de 834 ejemplos a partir de afirmaciones de QuUANTO y se implementaron dos evaluaciones complementarias.

El proyecto retoma el corpus UY22 introducido junto con ROUBERTa, pero explora una arquitectura y una tarea distintas. Mientras ROUBERTa es un encoder bidireccional entrenado con masked language modeling, **charruaLLM** es un decoder causal entrenado para generación. A pesar de esta diferencia, ambos trabajos ofrecen evidencia convergente sobre el valor de los datos de prensa uruguaya y la especialización de dominio.

charruaLLM obtuvo 26,62 % de exact matching frente a 4,68 % de **spanishLLM** sobre los 834 ejemplos combinados. También alcanzó una NLL media por palabra considerablemente menor: 4,1396 frente a 22,8143 en el conjunto completo. Las curvas de entrenamiento muestran asimismo una mejor loss de validación para **charruaLLM**: 3,0833 frente a 3,5004.

En conjunto, el experimento muestra que un modelo pequeño puede beneficiarse fuertemente de datos y tokenización adaptados al dominio. También evidencia que una evaluación responsable debe complementar la predicción exacta con medidas probabilísticas y explicitar las diferencias de tokenización, capitalización y posible solapamiento de datos.

5. Archivos producidos

- `final_dataset.csv`: dataset de 834 ejemplos.
- `exact_matching_results.csv`: predicciones y exact matching.
- `nll_results.csv`: NLL y probabilidades por palabra y token.
- `charruaLLM/best.pt`: mejor checkpoint de **charruaLLM**.
- `spanishLLM/best.pt`: mejor checkpoint de **spanishLLM**.
- `charruaLLM/losses.csv`: historial de entrenamiento.
- `spanishLLM/losses.csv`: historial de entrenamiento.
- `notebook.ipynb`: preparación, evaluación y visualización.

Referencias

- [1] Filevich, J. P., Marco, G., Castro, S., Chiruzzo, L. y Rosá, A. (2024). *A Language Model Trained on Uruguayan Spanish News Text*. Proceedings of the Second International Workshop Towards Digital Language Equality (TDLE), LREC-COLING 2024, pp. 53–60. <https://aclanthology.org/2024.tdle-1.5.pdf>
- [2] Hoffmann, J. et al. (2022). *Training Compute-Optimal Large Language Models*. arXiv:2203.15556. <https://arxiv.org/abs/2203.15556>
- [3] Liu, Y. et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. arXiv:1907.11692. <https://arxiv.org/abs/1907.11692>
- [4] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. y Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners*. <https://cdn.openai.com/better-language-models/language-models.pdf>
- [5] Rosá, A., Chiruzzo, L., Bouza, L. et al. (2022). *Overview of QuALES at IberLEF 2022: Question Answering Learning from Examples in Spanish*. <https://rua.ua.es/dspace/handle/10045/127433>
- [6] QuUANTO. *QuALES-derived Uruguay Assertions for Neural Training Objectives*. Repositorio del dataset. <https://github.com/McBruno712/QuUANTO>
- [7] Sennrich, R., Haddow, B. y Birch, A. (2016). *Neural Machine Translation of Rare Words with Subword Units*. Proceedings of ACL 2016, pp. 1715–1725. <https://aclanthology.org/P16-1162/>
- [8] Vaswani, A. et al. (2017). *Attention Is All You Need*. arXiv:1706.03762. <https://arxiv.org/abs/1706.03762>
- [9] Cañete, J. *Large Spanish Corpus*. Hugging Face Datasets. https://huggingface.co/datasets/josecannete/large_spanish_corpus
- [10] OPUS. *Open Parallel Corpus*. <https://opus.nlpl.eu/>